Software Development Project — G&G City · Lab Backlog Sprint 1

LAB P1: Product Owner + Lessons Data — Bassam Al-Dosari

Topic: Writing Product Content as Structured Data (JSON)

Learning outcomes

By the end of this lab, students will be able to:

- Order a backlog by value and dependency
- Write lesson content as valid JSON that code can read
- Accept or reject an increment against clear criteria

🎯 Problem Scenario

Every word the cat says and every command the learner types comes from your file. If a field name is wrong, C3's and S3's code breaks.

- **C3 reads `accepted`, `commonMistakes`, `matching` · S2 reads `title`, `background` · S3 reads `cat.intro`, `cat.task`**
- **P3 verifies every command before you mark it done**

Your files: `data/lessons.json, the Backlog`

Required Outputs

Build:

1. **Order the Sprint 1 backlog: blocking items first (`createCat`, `workshop.png`, `lessons.json`)**
2. **Write the workshop station (`git init`) in `lessons.json`**
3. **Add the next stations with the same structure**
4. **Review each lab's "Required Outputs" at the end of the sprint and accept or reject**

Code is given only for the hard parts. Parts marked 📝 **Note** are easy: follow the note and write them yourself. Test with **Live Server** in VS Code (`fetch` does not work when you double-click a file).

Model Answer:

Task 1: The backlog

> 📝 **Note (no code needed):** Put first what others are waiting for: C1's `createCat` (C2 needs it), S1's `workshop.png` (S2, S3 need it), and your `lessons.json` (C3, S2, S3 need it). Each item's acceptance criteria = the “Required Outputs” of its lab.

Task 2: The workshop station (the structure everyone depends on)

Keep every field name exactly as below. Wrap code inside Arabic sentences with the invisible marks `U+2066 … U+2069` so `.git` is not flipped.

```
"version": 1,
"matching": {
  "trim": true,
  "collapseSpaces": true,
  "caseSensitive": true,
  "note": "قبل المقارنة: احذف المسافات من البداية والنهاية، وحوّل المسافات المتكررة إلى
مسافة واحدة. الحروف الكبيرة والصغيرة مهمة لأن Git يفرّق بينها."
},
"stations": [
  {
    "id": "workshop",
    "order": 1,
    "title": "الورشة",
    "background": "scenes/workshop.png",
    "story": "أن يتتبّع ما Git القط يصل إلى ورشته الفارغة. فيها صندوق خشبي عادي، ولا يستطيع
بداخله حتى يحوّله القط إلى مستودع.",
    "lessons": [
      {
        "id": "workshop-init",
        "order": 1,
        "concept": "إنشاء مستودع جديد",
        "command": "git init",
        "accepted": [
          "git init",
          "git init ."
        ],
        "boxStateBefore": "empty",
        "boxStateAfter": "initialized",
        "boxLabel": ".git",
      "cat": { "intro": "…", "task": "…", "hint": "…", "success": "…", "failure": "…" },
      "commonMistakes": [ { "input": "git int", "feedback": "…" } ],
      "verifiedBy": "", "verifiedOn": ""
    } ]
  } ]
}
```

Task 3: Adding a station

> 📝 **Note (no code needed):** Copy the whole workshop object, change `id`, `order`, `title`, `background` and the lessons. `boxStateBefore` of a lesson must equal `boxStateAfter` of the one before it. After every edit, validate the file at **jsonlint.com** — one missing comma stops the whole game.

Task 4: Accept / reject

> 📝 **Note (no code needed):** Open the lab, go through the “Required Outputs” list, and test each point in the browser. If one fails, write in the task *what* failed and *how* to reproduce it; do not fix it yourself.

Submission

```
git pull
git add data/lessons.json
git commit -m "كتابة lessons.json الورشة لمحطة"
git push
```